

Pilarius: Multiplexing GPU Memory through Cyclic KV Restoration for Interactive Model Serving

Anonymous Author(s)

Abstract

Full-duplex interaction models process new input while generating output in ongoing sessions. For sessions that process time-aligned micro-turns at regular intervals, growing historical key-value (KV) state can exhaust GPU memory even when their combined execution takes only part of each period. KV offloading frees GPU memory, but on-demand restoration stalls computation and risks missing recurring update deadlines. We present PILARIUS, a serving system that multiplexes GPU memory across periodic interaction sessions through cyclic KV restoration. Its residency planner staggers session phases and coordinates partial eviction budgets and restoration times to spread host-to-device (H2D) demand across each period. Keeping the H2D link busy for longer allows more KV blocks to be restored per cycle and more GPU space to be reclaimed. PILARIUS executes this plan through a session manager that coordinates periodic updates and KV preparation. We pair a KV memory manager that maintains valid copies and manages GPU space with an asynchronous KV transfer engine for incremental backups and scheduled restores. This coordination expands effective KV capacity to support more concurrent sessions under the same history and periodic deadline requirements.

Placeholder. Measured session capacity and deadline-miss rates under matched service objectives and generation work, with workload and observation duration.

Keywords: KV cache, GPU memory management, model serving, full-duplex interaction, real-time inference, memory multiplexing, scheduling

1 Introduction

Recent models such as OpenAI’s GPT-Live and ByteDance’s Seeduplex support full-duplex human–AI interaction [3, 16]. By listening while speaking and adapting to interruptions, these models let users clarify or redirect a conversation without waiting for a response to finish. We refer to each ongoing interaction as an *interaction session*.

We study interaction sessions that advance through periodic updates, each processing new input and generating output as needed. For example, Moshi operates on 80 ms audio frames, while Thinking Machines Lab’s TML-Interaction-Small organizes input and output into 200 ms *time-aligned micro-turns* [5, 21]. A single response can span updates, allowing new input to influence generation before the response ends. The server must therefore preserve each session’s history across updates, including intervals when that session is not executing.

To reuse history across updates, the server caches attention keys and values (KV), whose footprint grows with the retained context. For MiniCPM-o 4.5, retaining ten minutes of continuous input and output at ten encoded audio positions and four generated text tokens per second requires an estimated 1.2 GiB of BF16 backbone KV [4, 17]. Under *full residency*, the KV for each session’s retained history stays on the GPU, occupying memory even when the session is not accessing it (Figure 1). As histories grow across concurrent sessions, their combined KV footprint can exhaust GPU KV capacity while their computation still fits within each period. We target this regime, where reclaiming the GPU space occupied by idle KV could allow the server to accommodate more sessions.

Offloading idle KV to host memory makes its GPU space available to other sessions. InferCept, for example, chooses among retaining, swapping, and recomputing KV for requests paused by external interactions [1]. For periodic interaction sessions, the next update’s scheduled release time is known in advance. Each update has a soft deadline at the start of the following period. If restoration starts only when an update is released, computation that depends on the evicted state must wait for the transfer, which can cause a deadline miss (Figure 2(a)). This timing information allows the server to restore KV before it is needed. A *tick* marks the start of each period; KV restoration is scheduled to finish before the next tick. Keeping idle KV in host memory between updates allows the server to reuse the same GPU space across sessions. We refer to this reuse as *memory multiplexing*.

Restoration scheduling must account for both transfer time and GPU space. Restorations share transfer bandwidth and require GPU memory for their destination buffers. Restoring too early occupies space that

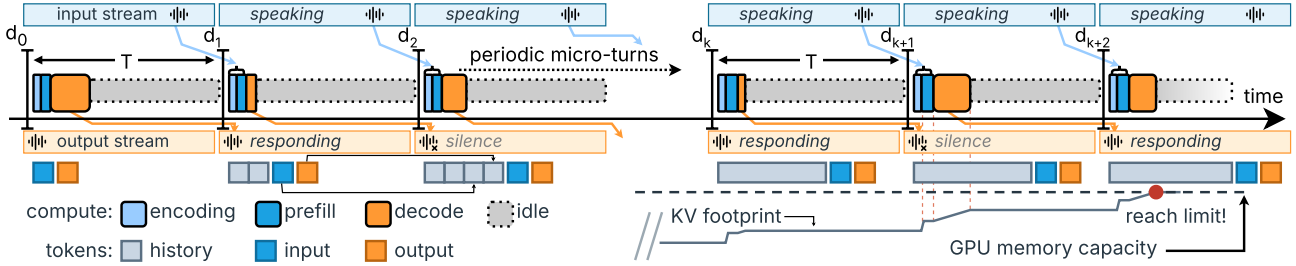


Figure 1. Historical KV remains allocated between updates under full residency. Each update encodes and prefills new input and may decode output for playback. Retained input and output extend the history, increasing the KV footprint toward the GPU capacity limit. The timeline and memory growth are illustrative.

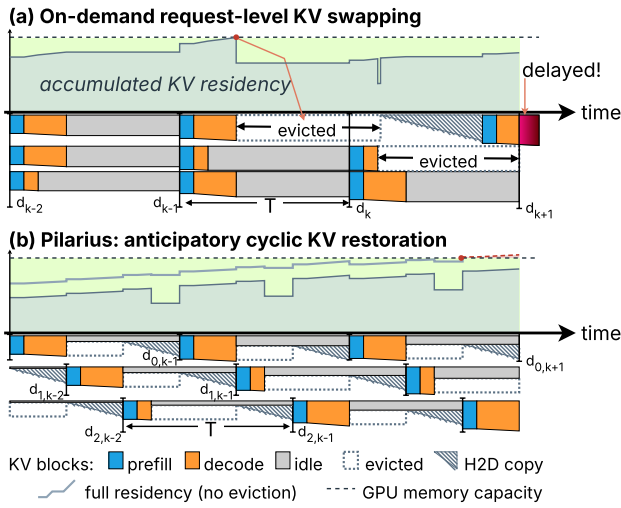


Figure 2. On-demand and scheduled KV restoration (schematic). (a) On-demand restoration delays computation. (b) Staggered sessions restore KV before their next ticks and reuse GPU space. Curves show allocated KV memory; the pale curve shows full residency. Hatching marks host-to-device (H2D) transfers, dotted outlines evicted KV, and gray idle resident KV.

other sessions could use; starting too late risks leaving KV unavailable at the next tick. Larger evictions free more space but require more data to be transferred back. The server must therefore coordinate how much KV to evict from each session, when to restore it, and when to schedule each session’s computation. The schematic in Figure 2 assumes that each restoration’s full destination is allocated before copying begins and counts that space in the memory curves from allocation onward.

We present PILARIUS, a serving system that multiplexes GPU memory across periodic interaction sessions through *cyclic KV restoration*. Within the timing flexibility allowed by the application, PILARIUS assigns each session a *phase*, or release offset within the period, to

spread H2D restoration demand over time. This keeps the link busy over more of the period, allowing more KV to be restored per period and supporting larger evictions. After a session completes an update, PILARIUS evicts part of its idle KV and schedules restoration to complete before the next tick. The plan coordinates phases, eviction amounts, and restoration times so that space released by one session can accommodate another’s restoration (Figure 2(b)). This coordination expands effective KV capacity to serve more concurrent sessions under the same history and soft deadline requirements.

To reuse a schedule as histories grow, PILARIUS plans for the KV required at each session’s configured maximum context length. The total retained context, including new input and generated output, must stay within this limit. The recurring schedule accounts for computation, transfers, and GPU allocation, while runtime operations act on the history actually retained. PILARIUS revises the plan when sessions join or leave.

Placeholder. Final contributions and measured results: identify the configurations where KV capacity limits concurrency while compute time remains; establish the benefit attributable to coordinating phases, eviction amounts, and restoration timing; report concurrency and deadline-miss rates against full residency and on-demand restoration under matched service objectives.

2 Background and Motivation

2.1 Periodic Work and Playback Pacing

An interaction session repeatedly processes new input while retaining history across updates. For a session with period T , each update is released at a tick and has a soft deadline at the next tick. Update latency is measured from its scheduled tick to completion, including any delay before submission. Once an update stops accessing its historical KV, that state can remain idle until the next update, even while the output is still playing. This interval creates an opportunity to reclaim GPU space while preserving the session’s history.

Table 1. Estimated backbone execution time for MiniCPM-o 4.5 at its configured context limit of 40,960 positions. Each GPU processes a batch containing as many sessions as its memory can hold, with one update per second. The central estimates assume 80% bandwidth efficiency and 25% compute efficiency; the sensitivity ranges vary these efficiencies over 40–80% and 15–40%, respectively.

GPU (capacity)	Memory BW (TB/s)	Batch size N_{mem}	Execution time (ms)	Efficiency sensitivity (ms)
A100 SXM (80GB)	2.04	10	241	234–476
H100 SXM (80GB)	3.35	10	141	141–282
RTX PRO 6000 Blackwell Server (96GB)	1.60	12	360	349–710
H200 SXM (141GB)	4.80	20	185	180–365
MI300X (192GB)	5.30	29	233	229–463

Speech illustrates why computing an update can take less time than playing its output. In models trained for time-aligned interaction, generation follows playback progress: each update typically produces content for the corresponding playback interval. For example, MiniCPM-o 4.5 trains on text and speech assigned to temporal chunks and uses Time-Aligned Interleaving to adjust generation to accumulated playback progress [4]. A long response therefore unfolds across updates, with new input incorporated along the way. Playback determines the useful pace of generation, while the hardware can compute that content much faster.

For example, a one-second update in MiniCPM-o 4.5 processes ten encoded audio positions and generates about three to four text tokens at normal speaking speed, excluding prompt and control positions [4]. Its public llama.cpp-omni implementation reports an audio prefill stage of about 21 ms and a backbone decoding cost of about 38 ms/token on an RTX 4090 in FP16 [9]. Combining these figures gives an estimated 114–152 ms of backbone decoding per second of speech, or 135–173 ms including the reported prefill stage. The reported stage timings omit speech synthesis and runtime overhead, and leave batch size and retained context length unspecified. Complete update time must therefore be measured at the target concurrency and history length.

2.2 Why KV Capacity Can Bind First

Playback pacing keeps typical generation per update small, while the retained history accumulates over successive updates. A session can therefore accumulate a large KV cache even when each update adds only a small amount of work and state. Keeping that cache on the GPU throughout the period consumes capacity even while the session is idle. Training does not impose a strict per-update token cap, and attention becomes more expensive as history grows. The available compute slack therefore depends on both the workload and its execution cost.

The relevant comparison is between the memory and computation required by all admitted sessions. Consider N sessions with a common period T , a maximum retained length of L positions per session during an update, and κ KV bytes per position, with no KV sharing. Let G_{KV} be the GPU capacity available for KV after other allocations. Let $C_N(L)$ be the total execution time required for one update from every session under the chosen grouping and batching policy. It includes input processing, generation, and runtime overhead, counting overlapping execution once and excluding idle gaps between scheduled releases. Ignoring allocation rounding, the full-residency capacity limit leaves execution slack when

$$L_{\text{mem}} = \frac{G_{\text{KV}}}{N\kappa}, \quad C_N(L_{\text{mem}}) < T. \quad (1)$$

The remaining aggregate execution budget is $T - C_N(L_{\text{mem}})$. Memory multiplexing must fit its additional execution and interference costs within this budget while also satisfying individual deadlines, restoration windows, and allocation constraints.

Placeholder. Measure complete periodic work at the full-residency capacity limit, including the distribution of per-update generation, context lengths, batching, and phase-dependent execution costs.

Table 1 uses the public backbone context limit of 40,960 positions to estimate how many sessions fit in GPU memory. The speech decoder’s separate 4,096-position limit is also included in the memory reservation [17]. The combined KV reservation is $K_{\text{sess}} \approx 5.86$ GiB per session; we assume another $R = 20$ GiB for weights and other allocations. Let M be the GPU memory budget, approximating nominal capacities as GiB. With $G_{\text{KV}} = M - R$, the memory-only session limit is $N_{\text{mem}} = \lfloor G_{\text{KV}}/K_{\text{sess}} \rfloor$. Each row assumes that all N_{mem} sessions form one batch near the context limit, with enough room in each history for the update’s new positions. This is the largest batch that fits within the

Table 2. Estimated fraction of each period used by backbone execution (%) across GPUs and models at their memory capacity limits. Each GPU cell reports the percentage followed by the batch size in parentheses; context limits and update periods depend on the model. All cells assume 80% bandwidth efficiency and 25% compute efficiency.

GPU (capacity)	MiniCPM-o 4.5	Qwen2.5-Omni 3B	Qwen2.5-Omni 7B	Moshi 7B
Context (positions)	40,960	32,768	32,768	3,000
Update period (ms)	1,000	1,000	1,000	80
A100 SXM (80GB)	24.1 (10)	34.5 (42)	24.3 (16)	58.5 (40)
H100 SXM (80GB)	14.1 (10)	15.0 (42)	10.9 (16)	35.6 (40)
RTX PRO 6000 (96GB)	36.0 (12)	55.9 (53)	38.1 (20)	92.7 (51)
H200 SXM (141GB)	18.5 (20)	24.1 (83)	17.2 (34)	47.5 (82)
MI300X (192GB)	23.3 (29)	27.9 (117)	20.2 (50)	59.7 (116)

KV memory budget. Device capacities and bandwidths follow vendor specifications [2, 12–15].

We estimate backbone prefill and decode by separating linear layers from attention and applying a roofline model to each component [11]. For component j , with byte traffic D_j and floating-point operation count O_j , the time estimate is

$$\hat{t}_j = \max\left(\frac{D_j}{\eta_B B_{\text{mem}}}, \frac{O_j}{\eta_F F}\right),$$

where B_{mem} is rated GPU memory bandwidth and F is a dense BF16 matrix-throughput reference. The central estimates in both tables assume bandwidth efficiency $\eta_B = 0.8$ and compute efficiency $\eta_F = 0.25$, without empirical calibration. Weights are read once for the whole batch, while the attention traffic includes each session’s own KV. For MiniCPM-o, each modeled update consists of a ten-position incremental prefill followed by four sequential decode forward passes. The component times are summed over these stages; parameter counts and traffic accounting appear in the supplementary material. The RTX estimate is higher than those for A100 and H100 because it combines lower memory bandwidth with a larger batch and therefore more total KV traffic.

Under these assumptions, estimated backbone execution time, including memory-access costs, is below one second in all five configurations. Table 1 also reports sensitivity to the assumed efficiencies for each GPU. KV reuse also affects the estimate: doubling the modeled attention traffic raises the RTX estimate to about 643 ms at the assumed efficiencies, or 1.01 s if bandwidth efficiency also falls to 50%. Staggering sessions can reduce batch sizes and increase weight traffic and total execution time, consuming part of the estimated margin. Establishing the complete-workload condition in Equation 1 requires measurements of usable memory, actual batching and traffic, input encoding, speech generation, and runtime overhead.

Variation across models. Table 2 extends the analysis to other model configurations. These are illustrative resource estimates. The added backbones are the 3B and 7B Qwen2.5-Omni Thinkers and Moshi’s temporal Transformer [5, 8, 18, 19]. For Qwen, we assume a one-second incremental update with 25 encoded audio positions and four text decode forward passes; the input rate follows its audio representation, while the update period and decode count are workload assumptions [20]. Moshi instead advances its temporal backbone once per native 80 ms frame. The supplementary material details auxiliary KV reservations and other memory budgets. As in Table 1, these estimates exclude input encoders and speech-output modules; for Moshi, the omitted stages include the depth Transformer. In the one-second scenarios, estimated backbone execution uses only part of the period across models with different sizes and KV footprints. All backbone estimates in Table 2 fit within their periods under the assumed efficiencies. For Moshi at its memory ceiling, the RTX estimate is about 74.2 ms, leaving only 5.8 ms of its 80 ms period for omitted stages. With compute efficiency fixed, reducing bandwidth efficiency to 70% or 50% raises that backbone estimate to 104.7% or 146.6% of the period. The compute slack available for memory multiplexing therefore depends on the workload and hardware together.

2.3 Bandwidth Limits on Memory Reuse

Periodic eviction and restoration allow idle KV to spend part of each period in host memory. Consider a steady-state schedule with a common period T , fixed history sizes, and no shared KV. Let e_i be the bytes evicted from session i and restored once per period, and let B_{eff} be the effective H2D bandwidth available for restoration. The total restoration volume must satisfy

$$E = \sum_i e_i \leq B_{\text{eff}} T. \quad (2)$$

The fraction of the transfer budget used is $E/(B_{\text{eff}}T)$; continuous transfers reach the ceiling $B_{\text{eff}}T$. The effective bandwidth must account for competing traffic, including incremental host backing. Each copy must fit within the interval between safe eviction and the next tick, with its GPU destination allocated before copying begins.

Why full eviction can fail. Evicting every session's entire history makes transfer demand grow with total retained state. Once this demand exceeds the budget in Equation 2, continuous transfers cannot restore every session's history within each period. A single large restoration can also exceed its available window or require more destination space than is free. For illustration, assume eight histories of 4.5 GiB, a one-second period, and 24 GiB/s of effective H2D bandwidth. Under these assumptions, full restoration requires 36 GiB per period, or 1.5 seconds of copying; evicting 2 GiB per session reduces this to 16 GiB and about 0.67 seconds. The partial-eviction plan must also fit the available copy windows and GPU space. Partial eviction allows the server to retain the state that cannot be restored in time.

From transferred bytes to usable capacity. Evicted KV saves GPU space from the moment it is freed until its restoration destination is allocated. For session i , let w_i be the duration of this interval averaged over its e_i evicted bytes, so $0 \leq w_i \leq T$. Let ΔG_{peak} be the reduction in peak allocation relative to full residency and $\overline{\Delta G}$ the average reduction over the period. Then

$$\begin{aligned} \Delta G_{\text{peak}} &\leq \overline{\Delta G} = \frac{1}{T} \sum_i e_i w_i \\ &\leq E \leq B_{\text{eff}}T. \end{aligned} \quad (3)$$

Thus, $B_{\text{eff}}T$ bounds the reduction in peak GPU KV allocation through memory multiplexing. The capacity gain depends on the duration and overlap of these intervals. Earlier destination allocation shortens these intervals. Staggering sessions lets one session's eviction free space for another's restoration, provided that overlapping allocations fit in GPU memory and execution fits within the available time.

3 System Design

PILARIUS turns the idle intervals identified in Section 2 into a recurring schedule for sharing GPU KV capacity. The schedule couples three decisions: when each session's update is released, how much of its KV to evict, and when to restore that state. By spreading restoration demand across the period, PILARIUS keeps the H2D link active for longer, increasing the KV volume that can be restored each cycle and hence the amount that can be evicted. Restoration targets the next tick so that the following update can use its period for computation. Meeting this

target removes a source of waiting; meeting the update's soft deadline also requires sufficient execution time after the tick.

3.1 Architecture and Execution Cycle

Figure 3 shows four logical components above the inference backend. The *residency planner* produces a plan at admission and revises it as resource demands change. The plan specifies session groups and phases, per-session eviction budgets, and restoration timing. The *session manager* follows this plan, maintaining periodic progress and coordinating KV preparation with request submission.

The *KV memory manager* carries out residency operations using the backend's allocation and reference state. It identifies safely reclaimable blocks, obtains restoration destinations, and tracks which GPU and host copies are valid. The *KV transfer engine* executes the resulting asynchronous copies and reports their outcomes. The inference backend schedules submitted work and performs model computation, including batch formation when used. Thus, the planner selects the residency schedule, while the other components enforce it against actual execution and memory state.

Consider a session in group C whose idle KV has been partially evicted. At the planned restoration time, the session manager requests its missing state. The memory manager obtains the destination space, and the transfer engine copies the state from host memory. Once all historical KV required by the next update is valid on the GPU, the memory manager reports readiness.

The session manager submits the update when its tick has arrived, its input is available, and any conflicting prior execution has finished. During computation, newly retained KV can enter the incremental host-backup path. After computation, safely backed and unreferenced ranges can be released again, supplying space for later restorations. This is the cyclic reuse illustrated in Figure 2(b).

3.2 Session Groups and Phase Assignment

Phase assignment uses the timing flexibility allowed by the application. For sessions with a common period T , PILARIUS divides the period into S evenly spaced slots. Session i assigned to slot j has phase $\phi_i = jT/S$, for $j = 0, \dots, S - 1$, and ticks

$$r(i, k) = t_0 + \phi_i + kT,$$

where t_0 is the common time origin and k indexes updates. Sessions in the same slot form a *session group*; they share a release phase but retain separate histories and eviction budgets. The group organizes residency scheduling, while the backend selects execution batches from eligible work. Uniform spacing distributes release

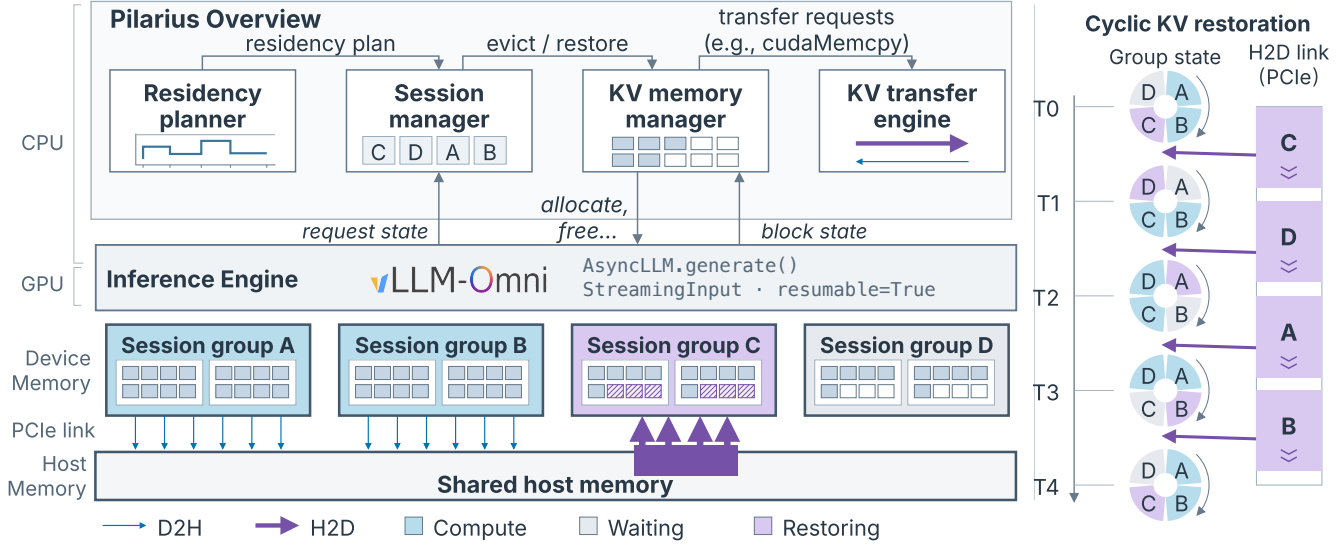


Figure 3. PILARIUS coordinates KV residency above the inference backend. The planner sets the residency schedule; the session manager follows it, and the memory manager and transfer engine perform safe reclamation and copying. Thin blue arrows show incremental device-to-host (D2H) backing during computation; thick purple arrows show scheduled H2D restoration. Solid, hatched, and empty KV blocks indicate valid state, allocated restoration targets awaiting completion, and missing state without an allocation. On the right, restorations for groups C, D, A, and B spread H2D demand over time to sustain link activity; white gaps indicate no H2D transfer. Group states and durations are schematic, and a group need not execute as one batch.

times without guaranteeing equal transfer demand or simultaneous execution within a group.

Trading batching efficiency for memory reuse.

Aligning every session to the same phase makes more work available for batching, but also concentrates demand for resident KV and restoration. Staggering groups spreads restoration demand over a longer part of the period, making more H2D service available for restoring evicted KV. Their eviction intervals supply space for one another, turning the additional restoration service into GPU memory reuse. Staggering can reduce batch sizes and increase total execution cost through additional model iterations and weight accesses. Assigning multiple sessions to each slot retains opportunities for batching. For N sessions in roughly balanced groups, each phase releases about N/S sessions, although actual batches depend on the backend and current readiness.

The goal is to preserve the service target while reducing peak memory use. Let C_{aligned} and C_{grouped} denote the complete periodic execution costs for the two release patterns at the same history length and workload. The extra cost $C_{\text{grouped}} - C_{\text{aligned}}$, together with transfer interference and scheduling delays, must fit within the original margin $T - C_{\text{aligned}}$. The short generation bursts described in Section 2.1 motivate this tradeoff; admission must check execution costs under the candidate grouping

and each session’s deadline. Initial phase-alignment waiting is recorded separately, and later submission delays do not move the ticks.

3.3 Joint Residency Planning

The planner sizes each session’s KV demand at the maximum context length configured in the model and backend. Input and generation must respect that limit, including all positions retained by the reference history policy. This approach accounts for future state growth when selecting the recurring schedule. It can admit fewer currently short sessions, but allows the plan to be reused as their histories grow within the limit.

The planner combines this demand with session periods, execution costs under the configured workload and candidate groupings, GPU KV capacity, host capacity, and effective transfer service. Its output couples group assignments, per-session eviction budgets, and restoration times within the recurring schedule. Planning at maximum context does not require every session’s KV to remain resident simultaneously; runtime copies and allocations cover only existing state. A candidate must fit the complete workload’s compute budget and each session’s deadline, as well as the time and space constraints below. As Equation 3 shows, using more transfer bandwidth is useful only if the resulting eviction intervals reduce peak GPU allocation.

Eviction budgets. An eviction budget e_i limits the bytes of safely recoverable KV released from session i in a cycle. It need not cover the entire history. As Section 2.3 shows, full eviction becomes infeasible when the retained state exceeds the available restoration budget. Partial eviction keeps the excess resident and gives the planner control over each restoration’s duration and destination size. A useful budget must free enough memory to matter while leaving a feasible return path.

Restoration timing and space. For the restoration preceding tick $r(i, k + 1)$, let u_i be the planned trigger time and $\widehat{B}_i$ the estimated effective bandwidth available to that restoration. A budget check takes the form

$$u_i + \frac{e_i}{\widehat{B}_i} + \delta_i \leq r(i, k + 1),$$

where δ_i accounts for queueing, completion handling, and a safety margin. These estimates must describe a feasible shared transfer schedule: independent estimates that each assume the full link bandwidth are insufficient. Equation 2 supplies an aggregate necessary condition, while individual recovery windows constrain the order and timing.

Space must be available when a restoration obtains its destination, even though the copy has not finished. The planner therefore accounts for each entire destination from allocation until release, together with resident history and newly generated KV; shared physical blocks count once. Total allocation must remain within G_{KV} throughout the recurring schedule. An earlier restoration gives the transfer more time but shortens the interval during which its state frees GPU space. The plan must consequently match releases by preceding groups to allocations for upcoming groups, accounting for state and host-copy storage at the configured context limits. Algorithm 1 constructs a feasible recurring plan when a new session arrives. It keeps the active slot grid and existing phases, and tries slots in increasing order of population, initial alignment delay, and slot index. For each placement, it considers an ordered, finite set of eviction-budget and restoration-lead profiles calibrated for the workload and backend. The evaluated conservative policy fixes one profile; a larger candidate set changes the search, not the feasibility conditions. Within a candidate, restorations follow their planned trigger times, with a stable session order for ties. The search returns the first feasible plan and makes no optimal-capacity claim.

The time check uses execution costs for the candidate group sizes, including backup and transfer interference, and checks both shared service and individual deadlines. The allocation scan charges each complete restoration destination when it is allocated, including time spent queued or ready before its tick. It credits an eviction only after execution and backing allow safe release; a

Algorithm 1 Admission with a recurring residency plan

Require: New session x , active plan P , resources R , ordered profiles $\mathcal{C}$

- 1: $D \leftarrow$ KV demands at each session’s maximum context
- 2: **for** s in slots ordered by population, alignment delay, and index **do**
- 3: **for** c in $\mathcal{C}$ **do**
- 4: $Q \leftarrow$ extend P with x in s using profile c
- 5: Preserve existing phases; order restorations in Q
- 6: **if** host storage for D exceeds capacity **then**
- 7: **continue**
- 8: **end if**
- 9: **if** group execution or shared transfer windows do not fit **then**
- 10: **continue**
- 11: **end if**
- 12: $E \leftarrow$ allocation and safe-release events for (Q, D)
- 13: Include cyclic boundaries and the transition from P
- 14: $m \leftarrow$ initial physical occupancy, including reservations
- 15: fits $\leftarrow (m \leq R.\text{gpuCapacity})$
- 16: **for** a in E ordered by time, allocations before releases at ties **do**
- 17: $m \leftarrow m + \text{allocationDelta}(a)$
- 18: **if** $m > R.\text{gpuCapacity}$ **then**
- 19: fits $\leftarrow$ false; **break**
- 20: **end if**
- 21: **end for**
- 22: **if** fits **then**
- 23: **return** Q for revalidation at the activation boundary
- 24: **end if**
- 25: **end for**
- 26: **end for**
- 27: **return** REJECT; retain P

newly admitted session receives no credit for a host copy that does not yet exist. The scan covers wraparound between periods and current allocations and references during activation. These are feasibility predictions under the calibrated cost estimates; runtime readiness and reference checks remain mandatory. Section 3.5 describes activation and rejection when revalidation fails.

3.4 Safe Eviction and Scheduled Restoration

Runtime execution enforces the plan without treating predicted availability as permission to access or free a block. The memory manager distinguishes physical allocation, valid content, and references from computation

or copying. Three steps govern the transition from idle state to the next update.

Backing and releasing state. Incremental host backing runs during inference as newly retained blocks become safe to copy. Each session can accumulate such blocks and issue asynchronous D2H copies without waiting for its entire update to finish. Completed host copies of unchanged history are reused across cycles, so later backups cover newly retained state rather than repeatedly copying the whole history. The memory manager extends valid host coverage only after a successful copy.

Backing up KV does not itself reclaim GPU space. After an update completes, the memory manager checks the ranges selected for eviction against the budget and restoration plan. A block can be released only if its host copy is valid and all execution, sharing, and transfer references permit reclamation. If a copy or reference is still outstanding, reclamation waits; only space actually released becomes available for reuse. These checks preserve the selected history while making its idle GPU allocation available to other sessions.

Allocating and copying. At the planned restoration time, the memory manager checks which required ranges remain missing and confirms their host coverage. It allocates the complete missing destination for one session before submitting H2D copying, potentially using space released by another group. The transfer engine queues H2D copies in the plan’s session order; incremental D2H copies may overlap them. Several sessions can have complete destinations allocated while their copies await execution or completion. This choice avoids reserving an entire group’s destinations before its first session can restore, at the cost of additional allocation decisions and possible waiting between sessions.

If the complete target cannot be allocated, the operation waits without holding a partial destination or reserving space for later sessions. Resource changes trigger retries; the runtime reports a timing risk when waiting threatens the planned recovery window. It does not bypass the blocked session under the current plan. This policy can cause head-of-line blocking, so the planner must check the resulting order and allocation timeline. Every allocated destination counts toward capacity immediately, including completed restorations waiting for their ticks.

Resuming computation. On successful transfer completion, the memory manager validates the operation’s identity and publishes the corresponding GPU ranges as valid. It reports KV readiness only when every historical range required by the update is available. Readiness is per session: a restored session need not wait for the rest of its group before becoming eligible for submission.

Cancellation, duplicate requests, or plan changes cannot allow an old completion to publish state for a different operation or session. Readiness checking and acquiring execution references must be coordinated with eviction so that submitted work retains access to its state.

KV readiness and the tick are separate conditions for submission. An early restoration waits for the tick and the update’s other execution conditions; a tick that arrives before restoration finishes leaves dependent computation waiting. On-demand restoration can recover missing state, but the resulting delay remains part of latency measured from the original tick. A missed timing target never permits computation to read unfinished KV.

3.5 Plan Reuse and Revision

The planner reuses a plan as histories grow within their configured limits and revises it when sessions arrive or depart. Context growth alone does not require periodic replanning. Existing sessions keep their phases and the current slot grid; the revision changes future eviction budgets and restoration timing and places new sessions on that grid.

To activate a candidate, the runtime pauses new restoration allocations and lets operations with allocated H2D targets finish or settle safely. It then rechecks the candidate against current allocations and references, including the time needed for the transition. If the check succeeds, it advances the plan version and replaces only operations that have not started. Computation and D2H backing can continue across the transition, with their references and completion events preserved. This boundary simplifies switching but can add waiting to the restoration queue.

If a candidate cannot accommodate a new session, admission is rejected and the existing plan remains in use. Blocked allocations wait for resource events, and copy or execution errors follow the existing error paths. Updates retain their original ticks, so recovery waits remain visible in the measured completion latency.

3.6 Backend Integration

Integrating PILARIUS requires access to the execution and KV state used by the preceding checks. The backend must support request submission and execution-state notifications, identify KV blocks and their valid ranges, and coordinate allocation, references, and safe release with the memory manager. It must also expose when newly produced KV is safe to copy and respect copy-completion dependencies before using restored state. PILARIUS uses the backend’s authoritative allocation and reference state so that residency management and model execution agree on which physical blocks are available.

These interfaces allow the backend to retain control of execution scheduling and batching. The session manager submits eligible updates and consumes execution events; it does not drive each model step or require a group to execute as one batch. A backend’s support for persistent sessions supplies part of this interface, but KV lifecycle coordination is also required.

Placeholder. Verified implementation: backend and version, supported KV operations, synchronization with execution, and implemented planning and recovery paths relative to the design above.

4 Evaluation

The evaluation tests whether lower peak GPU KV residency translates into more periodic work completed on time, and quantifies the latency and transfer costs. Three experiments examine capacity, service under session arrivals and departures, and the contributions of phase assignment and restoration timing.

Placeholder. Results are pending. Report the evaluated model, input and output workload, platform, software versions, and verified implementation coverage.

4.1 Methodology

Compared systems. We compare three configurations: GPU full residency, on-demand host restoration, and PILARIUS. Full residency retains the same reference history on the GPU and rejects new sessions beyond a static admission limit determined from the common maximum context and independent calibration. It uses naturally staggered session clocks and a suitable backend batching configuration. On-demand restoration uses the same host-copy and partial-eviction implementation as PILARIUS, but restores missing KV when current work needs it. This configuration isolates the contribution of advance restoration within the shared implementation. PILARIUS combines residency planning, phase assignment, and restoration before the next tick. Each system uses its own admission limit under the same resource and service constraints; rejected sessions are counted and are not automatically retried.

Workload controls. Comparisons fix the model and precision, maximum context, history-retention policy, period, original input content and timestamps, and per-session generation settings. At equal concurrency, we also check the actual input, generation, and retained history. Capacity comparisons hold offered work fixed and measure how much each system completes. Input streams use reproducible content and per-session offsets, avoiding identical streams that could inflate cross-session reuse. Initial histories and measurement durations are matched, and retained input and generation positions

must remain within the context limit. Backend settings and resource budgets are recorded, including any differences required by the mechanisms. Each data point starts a fresh worker; independent calibration is separate from the repeated measurement runs.

Completion and latency. Let $c(i, k)$ denote completion of model generation associated with update k of session i , and let $r(i, k)$ be its planned tick. An update is on time when $c(i, k) \leq r(i, k + 1)$; its periodic completion latency is $c(i, k) - r(i, k)$. We report on-time, late, and unfinished updates alongside latency distributions and offered, admitted, rejected, and completed sessions. Rejected sessions are unserved demand, rather than deadline misses among admitted sessions. Delayed submission does not move the original deadline. Initial phase-alignment waiting, input backlog, and delivery latency from the original input clock remain visible. RPC returns and previously buffered output do not establish completion of new model work. Failed or timed-out runs retain their completed and unfinished work under the common observation interval.

Resources and validation. Peak KV occupancy counts physical GPU blocks unavailable to other sessions, including restoration destinations from allocation onward; shared blocks count once. We also record host-copy storage, H2D and D2H bytes, restoration waits, and actual batch shapes. Detailed traces at selected points distinguish allocation, copying, and scheduling waits, with profiling overhead checked separately from the main runs. Before performance measurements, short checks compare KV content and model results across eviction and restoration and verify session termination and resource release. The checks record controls for nonde-terminism and distinguish state correctness from timely completion.

4.2 Capacity and Service Cost

The first experiment compares all three systems at five concurrency points spanning low load, the full-residency boundary, and higher offered concurrency, with three independent runs per point. Each run uses a fixed session population with staggered input clocks and records histories growing toward the common context limit. The same runs provide equal-concurrency resource comparisons, capacity curves, and measurements of how costs change with context length. Low-load points expose overheads when little additional capacity is needed.

Long-history initialization can shorten a controlled comparison when its state fits during setup. It uses the same reconstructible history and excludes initialization output from service measurements. If the highest concurrency cannot initialize all histories while resident, the entire sweep starts from a common shorter history and

grows through real input and generation over a matched duration. The comparison records actual context lengths and preserves room for subsequent work rather than silently truncating history.

Interpreting capacity. Near the full-residency limit, joint allocation and execution observations test whether KV capacity prevents additional service while the aggregate periodic work still leaves compute budget. The additional work completed by PILARIUS then tests whether that budget remains usable after restoration and changes in batching. A protected baseline may continue serving admitted sessions promptly while rejecting additional sessions. A capacity gain requires PILARIUS to complete additional sessions' work on time, together with a report of the associated latency and resource costs. We retain the full curves rather than classify configurations using an added deadline-miss tolerance.

Placeholder. Report admitted and rejected sessions, on-time/late/unfinished updates, completion latency, peak GPU KV occupancy, host storage, transfer traffic, and variation across runs. Show the time and context-length trajectories at the capacity boundary.

4.3 Dynamic Session Arrivals and Departures

The second experiment compares full residency and PILARIUS using the same open-loop session schedule. Successive session interarrival times are exponentially distributed with rate λ . Each session lasts for a fixed duration τ and replays its assigned input stream at the original rate. The schedule records each session's arrival, duration, content identifier, and content offset, together with the random seed. Neither new arrivals nor ongoing input wait for earlier work to complete. Planned and actual send times expose delays introduced by the load generator.

Natural staggering. For a session arriving at time s_i , input for its first period becomes available at $s_i + T$, followed by periodic input at $s_i + 2T, \dots$. These session-relative clocks retain their fractional offsets rather than being rounded to a shared global tick. Random arrivals therefore produce naturally staggered phases and a mixture of session ages. PILARIUS receives the same input segments at the same original times and may buffer them for its assigned phases within the application's timing flexibility. Its initial alignment wait and input-to-result delay are included in the measurements. The planner does not receive the generator's future arrivals or departure times.

Load and observation. We use three arrival rates covering low load, the full-residency boundary, and higher load, with three independently seeded schedules per rate replayed across both systems. The product $\lambda\tau$ provides

a reference for average offered concurrency after startup, rather than the number actually admitted. The arrival window spans at least two session lifetimes to include departures and replacement arrivals. After new arrivals stop, existing sessions continue to their scheduled input end; remaining work is accounted for under the common termination rule. Draining is reported separately from throughput during offered load, and context-limit termination is distinguished from execution failure.

Placeholder. Report admission and actual completion under the three arrival rates, with latency and unfinished work. Plot active sessions, input backlog, context growth, and KV occupancy over time, including costs during plan transitions.

4.4 Mechanism Ablations

The third experiment uses one or two representative boundary points from the capacity sweep. Two additional variants reuse PILARIUS's implementation: natural phases with advance restoration, and prefetching after input submission with the assigned phases. Together with the three main systems, these give five configurations in total. Matched main-system runs are reused where their workloads and controls agree.

Phase assignment. The first comparison holds concurrency, history, eviction budgets, host-copy policy, and advance restoration fixed, and compares natural session phases with PILARIUS's assigned phases. It measures whether active phase assignment improves peak allocation and periodic completion beyond the staggering already provided by independent session clocks. Actual batch shapes and execution costs capture the cost of changing those phases.

Restoration timing. The second comparison fixes phases and the other residency settings, and changes only the trigger: current demand, input submission, or advance restoration for the next tick. For each update, traces associate destination allocation, copy start and completion, tick, execution start, and generation completion. These events test whether earlier restoration reduces dependent work's wait and how much GPU space it occupies in advance.

Placeholder. Report the two paired comparisons: peak KV occupancy, completion counts and latency, restoration waits, and the event timelines that explain them.

5 Discussion and Limitations

The design's benefit depends on how much idle state can be reclaimed and restored within the service budget. Three conditions limit that opportunity: application timing, maximum-context planning, and the resources available for recovery.

Application timing and phase control. An application may expose future ticks while fixing its release phases. Advance restoration can use those ticks, but phase assignment then provides no additional control over demand. With different periods or unequal histories, evenly spaced phases also need not produce evenly distributed transfer or compute demand. Furthermore, a predicted tick does not guarantee that new input arrives on time. Late input and execution backlog reduce the useful idle interval, and restoring historical KV cannot remove those sources of delay.

Planning at maximum context. Sizing session state at the configured context limit allows the schedule to be reused as histories grow. It can reserve more capacity than currently short sessions need and therefore admit fewer of them. The context limit bounds retained state; measured completion latency still depends on the workload, batching, and transfer costs.

Resource balance and topology. A small retained history leaves little memory to reclaim, and a short interval between uses leaves little time to restore it. Effective transfer bandwidth, host-copy availability, and batching efficiency determine the useful eviction budget. When restoration spans multiple devices, the model must track destination capacity on each device and contention along each transfer path, including shared host-memory channels and interconnects. The effective bandwidth in Equation 2 must reflect that topology; summing nominal per-device bandwidths can overestimate available service.

Future work: recomputation. The current residency design restores state through H2D transfers. A further planning dimension would choose which evicted ranges to reconstruct from retained inputs and which to load from host memory. This extension would require backend replay support, space for reconstructed KV and temporary work, and compute slack across all admitted sessions. Recovery would still need to finish before use while preserving the computation budget for normal updates. The joint transfer and recomputation policy and its evaluation remain future work.

6 Related Work

Periodic interaction serving. Metronome studied memory exhaustion in periodic interaction serving and combined an in-engine KV window with deadline-aware admission [10]. Its window retains recent tokens and attention sinks, bounding per-session state, and its evaluation used phase-staggered input streams. PILARIUS addresses where the application’s selected history resides between updates and when missing state returns to the GPU. Its scheduling decisions couple phases, eviction

amounts, and restoration destinations under shared resource constraints. Residency scheduling can apply to full or bounded histories; comparisons hold the retention policy fixed to separate residency effects from changes in model history.

State recovery during interruptions. InferCept managed requests paused by interactions with tools or external environments [1]. It estimated interruption durations and selected among preserving, swapping, and discarding KV for later recomputation, with pipelining and chunking to limit interference. It thus accounted for both idle memory occupancy and recovery cost. For periodic interaction sessions, planned ticks supply recurring restoration targets, and applications may also permit phase assignment. PILARIUS uses those inputs to coordinate repeated recovery and destination allocation across sessions.

Combining loading and recomputation. Cake prepared cached prefixes by concurrently loading some KV chunks and recomputing others [6]. Its schedule rebuilt earlier positions while loading later ones, exploiting the increasing attention cost of later chunks. This work supports using computation to supplement loading when I/O is limiting. PILARIUS’s current design schedules H2D recovery across periods; incorporating recomputation into that schedule remains the extension described in Section 5.

Execution scheduling and KV allocation. Orca introduced iteration-level scheduling so that batch membership could change as requests arrived and finished [22]. PagedAttention mapped logical KV blocks to noncontiguous physical storage, reducing fragmentation and enabling sharing; vLLM also evaluated swapping and recomputation for preempted requests [7]. These techniques provide efficient execution and memory allocation within a serving engine. PILARIUS operates across successive uses of retained state, coordinating its residency with future ticks while leaving model execution and batching to the backend.

Latency-constrained serving. DistServe separated prefill and decoding onto different GPUs and allocated resources against time-to-first-token and time-per-output-token objectives [23]. It addressed interference between execution stages with different latency requirements. Our service objective measures completion against recurring session ticks and includes any waiting for retained state. Both approaches evaluate the load served within a latency objective, which is also the basis of the capacity metric in Section 4.1.

7 Conclusion

PILARIUS uses cyclic KV restoration to share GPU capacity across periodic interaction sessions while preserving their selected histories. Its residency plan couples phase assignment, partial eviction, and restoration timing so that space released by idle state can prepare upcoming updates. The central constraint is both temporal and spatial: a restoration needs enough transfer time and a complete GPU destination before copying begins. Planning sizes session state at the configured maximum context and reuses the schedule as histories grow within that limit.

Placeholder. Conclude with measured session-capacity gains and their latency and resource costs under the evaluated workloads, service objective, and observation durations.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [2] Advanced Micro Devices. 2025. AMD Instinct MI300X Accelerator Data Sheet. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-data-sheet.pdf>
- [3] ByteDance Seed. 2026. Introducing Seed Full-Duplex Speech LLM: Attentive Listening, Robust Interference Suppression, Enabling More Natural Interaction. <https://seed.bytedance.com/en/blog/introducing-seed-full-duplex-speech-llm-attentive-listening-robust-interference-suppression-enabling-more-natural-interaction>
- [4] Junbo Cui, Bokai Xu, Chongyi Wang, Tianyu Yu, Weiyue Sun, Yingjing Xu, Tianran Wang, Zhihui He, Wenshuo Ma, Tianchi Cai, Jiancheng Gui, Luoyuan Zhang, Xian Sun, Fuwei Huang, Moye Chen, Zhuo Lin, Hanyu Liu, Qingxin Gui, Qingzhe Han, Yuyang Wen, Huiping Liu, Rongkang Wang, Yaqi Zhang, Hongliang Wei, Chi Chen, You Li, Kechen Fang, Jie Zhou, Yuxuan Li, Guoyang Zeng, Chaojun Xiao, Yankai Lin, Xu Han, Maosong Sun, Zhiyuan Liu, and Yuan Yao. 2026. MiniCPM-o 4.5: Towards Real-Time Full-Duplex Omni-Modal Interaction. arXiv:2604.27393 [cs.CL] doi:10.48550/arXiv.2604.27393
- [5] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. 2024. Moshi: A Speech-Text Foundation Model for Real-Time Dialogue. arXiv:2410.00037 [eess.AS] doi:10.48550/arXiv.2410.00037
- [6] Shuwei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. 2025. Compute Or Load KV Cache? Why Not Both? arXiv:2410.03065 [cs.LG] doi:10.48550/arXiv.2410.03065
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, 611–626. doi:10.1145/3600006.3613165
- [8] Kyutai. [n.d.]. Moshi: Model Loading and Configuration. <https://github.com/kyutai-labs/moshi/blob/e6a55d2722a65870ef52a6c9f6ecfc0e90f38362/moshi/moshi/models/loaders.py>
- [9] llama.cpp-omni Contributors. 2026. llama.cpp-omni: Architecture and Performance Benchmarks. <https://github.com/tc-mb/llama.cpp-omni/blob/3dfebb53aaa1ae9cdf5e71ec4db1db65effdfad/README.md>
- [10] Jiaying Meng and Bojie Li. 2026. Metronome: Bound the Cache, Keep the Beat for Real-Time Interaction Model Serving. arXiv:2607.02640 [cs.SD] doi:10.48550/arXiv.2607.02640
- [11] NVIDIA. [n.d.]. GPU Performance Background User's Guide. <https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html>
- [12] NVIDIA. [n.d.]. NVIDIA H100 Tensor Core GPU Datasheet. https://i.dell.com/sites/csdocuments/App-Merchandizing_Documents/en/us/h100-gpu.pdf
- [13] NVIDIA. [n.d.]. NVIDIA H200 GPU: Specifications. <https://www.nvidia.com/en-us/data-center/h200/>
- [14] NVIDIA. [n.d.]. NVIDIA RTX PRO 6000 Blackwell Server Edition: Specifications. <https://www.nvidia.com/en-us/data-center/rtx-pro-6000-blackwell-server-edition/>
- [15] NVIDIA. 2021. NVIDIA A100 Tensor Core GPU Data Sheet. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf>
- [16] OpenAI. 2026. Getting Started with GPT-Live. OpenAI API Documentation. <https://developers.openai.com/api/docs/guides/live>
- [17] OpenBMB. 2026. MiniCPM-o-4.5 Model Configuration. <https://huggingface.co/openbmb/MiniCPM-o-4.5/blob/503e754207c94da6bb26850b4469f367c9ea3582/config.json>
- [18] Qwen Team. 2025. Qwen2.5-Omni-3B Model Configuration. <https://huggingface.co/Qwen/Qwen2.5-Omni-3B/blob/f75b40e3da2003cdd6e1829b1f420ca70797c34e/config.json>
- [19] Qwen Team. 2025. Qwen2.5-Omni-7B Model Configuration. <https://huggingface.co/Qwen/Qwen2.5-Omni-7B/blob/ae9e1690543ffd5c0221dc27f79834d0294cba00/config.json>
- [20] Qwen Team. 2025. Qwen2.5-Omni Technical Report. arXiv:2503.20215 [cs.CL] doi:10.48550/arXiv.2503.20215
- [21] Thinking Machines Lab. 2026. Interaction Models: A Scalable Approach to Human-AI Collaboration. *Thinking Machines Lab: Connectionism* (May 2026). doi:10.64434/tml.20260511
- [22] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [23] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>